ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

CHƯƠNG 3
TẦNG VẬN CHUYỂN

NHẬP MÔN MẠNG MÁY TÍNH



TRẦN THỊ DUNG

Khoa Mạng máy tính và Truyền thông



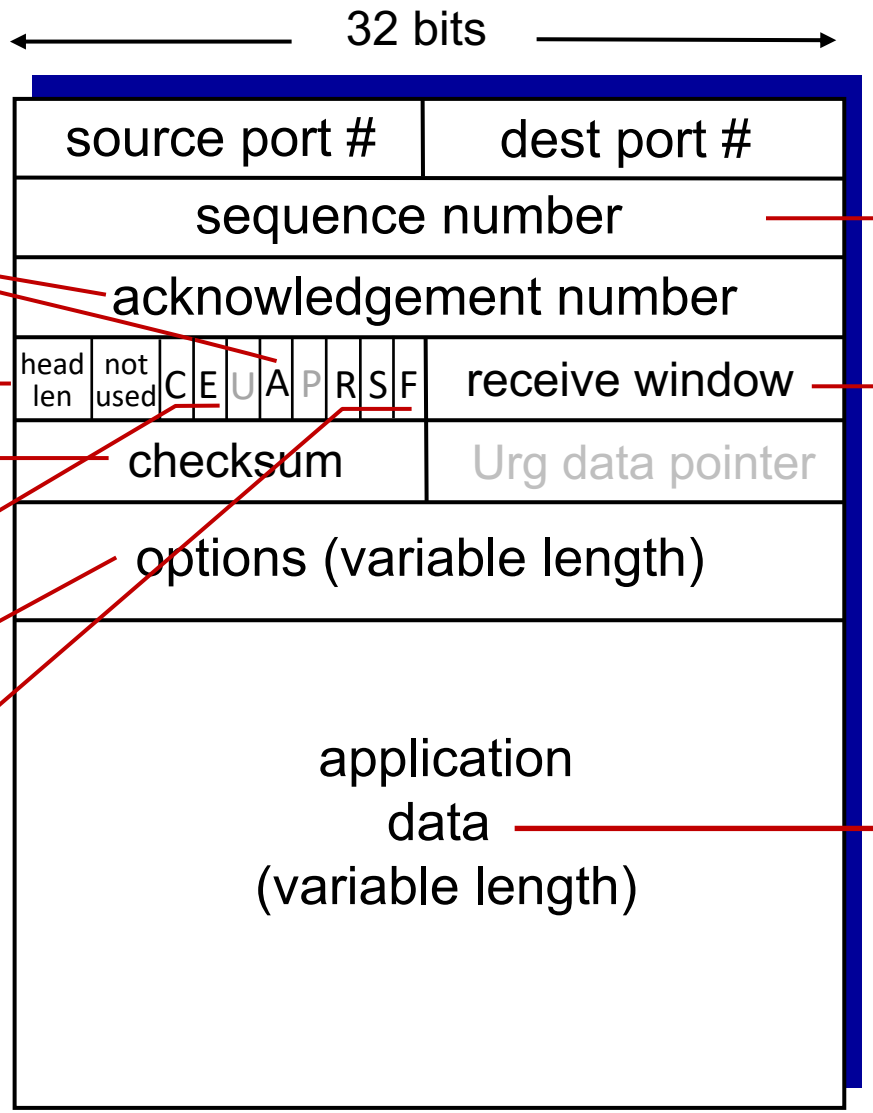
Nội dung

- Các dịch vụ tầng vận chuyển
- Multiplexing and demultiplexing
- UDP
- Nguyên lý truyền tin cậy
- TCP
- TCP - Điều khiển tắc nghẽn
- Sự phát triển của các tính năng của tầng vận chuyển

TCP: tổng quan

- Tin cậy, dữ liệu đúng thứ tự theo byte
- "full duplex data":
  - Luồng dữ liệu 2 chiều trong cùng một kết nối
  - MSS: maximum segment size (độ dài tối đa của 1 segment)
- ACKs tích lũy
- Cơ chế pipelining:
  - Điều khiển tắc nghẽn và điều khiển luồng với "window size"
- Hướng kết nối:
  - handshaking (trao đổi các thông điệp thiết lập và đóng kết nối)

Cấu trúc của TCP segment



ACK: seq # of next expected byte; A bit: this is an ACK

length (of TCP header)

Internet checksum

C, E: congestion notification

TCP options

RST, SYN, FIN: connection management

segment seq #: counting bytes of data into bytestream (not segments!)

flow control: # bytes receiver willing to accept

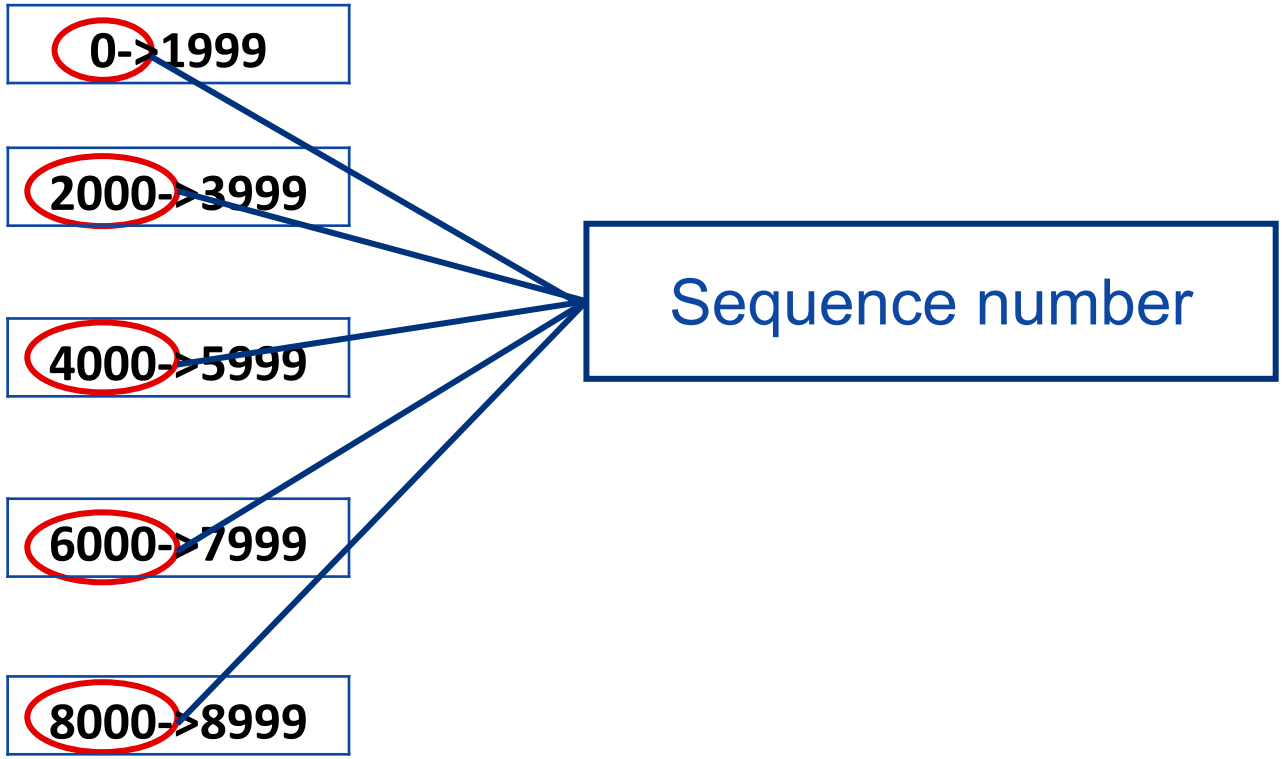
data sent by application into TCP socket

TCP: Truyền tin cậy

- Truyền theo thứ tự: Sequence number, ACK
- Truyền lại:
  - Segment lỗi: ACK
  - Segment bị mất: timeout

TCP sequence numbers, ACKs: ví dụ

- Thông điệp tầng ứng dụng: 9000 bytes (byte #0 -> byte #8999)
- Mỗi segment chứa tối đa 2000 bytes (MSS)
- => 5 2000-byte segments



TCP sequence numbers, ACKs: ví dụ

- Thông điệp tầng ứng dụng: 9000 bytes (byte #0 -> byte #8999)
- Mỗi segment chứa tối đa 2000 bytes (MSS)
- => 5 2000-byte segments

0->1999	2000->3999	4000->5999	6000->7999	8000->8999

Sender - Sequence number

Receiver - ACK

Seq = 0, length = 2000B

ACK = 2000

Tôi nhận được đến byte #1999, tôi cần từ byte #2000 trờ đi

Tôi sẽ gửi từ #2000

S = 2000, length = 2000B

ACK = 4000

S = 4000, length = 2000B

ACK = 6000

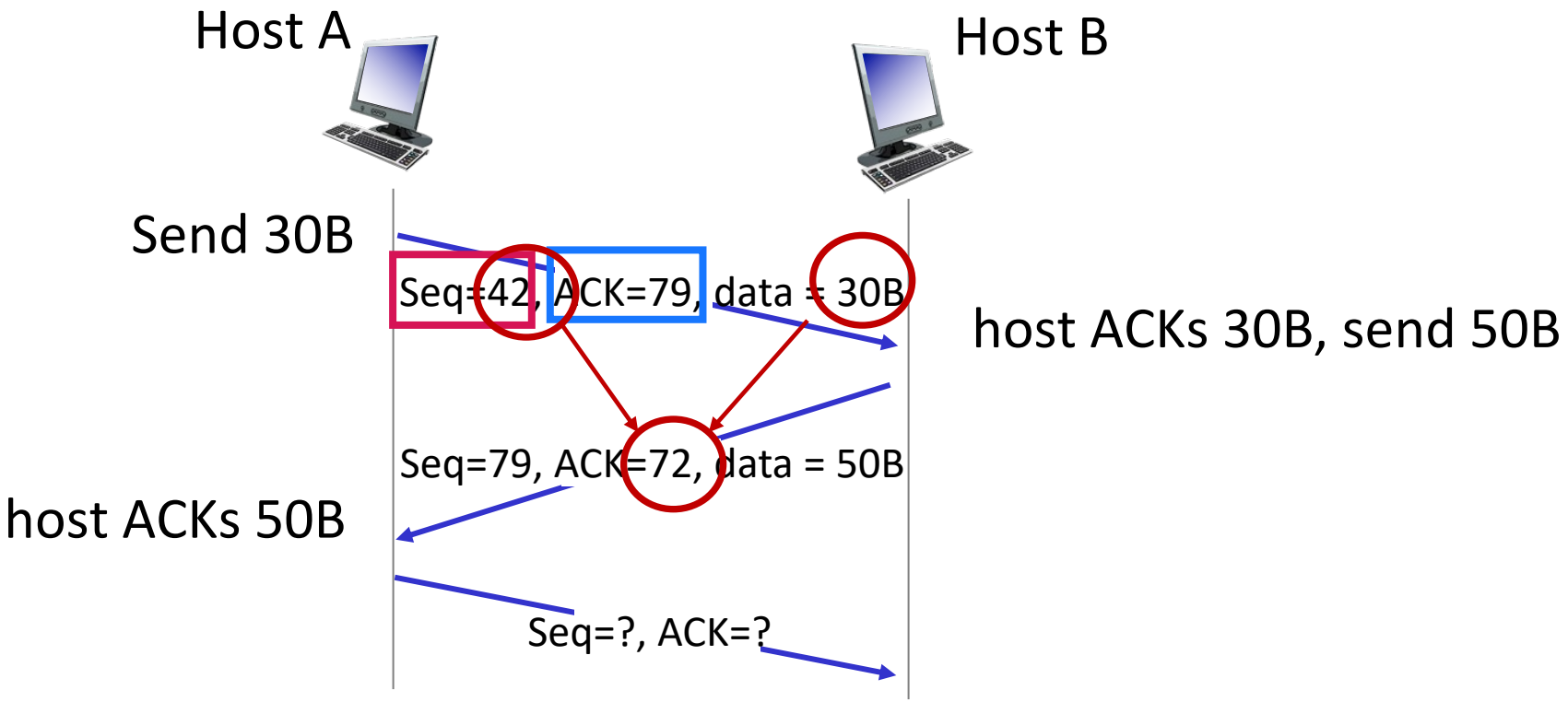
S = 6000, length = 2000B

ACK = 8000

S = 8000, length = 1000B

ACK = 9000

TCP sequence numbers, ACKs



TCP round trip time, timeout

Q: TCP xác định thời gian timeout thế nào?

- Phải lâu hơn RTT, nhưng RTT khác nhau sau mỗi lần gửi!
- *Nếu quá ngắn:* gửi lại không cần thiết
- *Nếu quá lâu:* chậm phản ứng với việc mất segment

Q: Làm thế nào để dự đoán RTT?

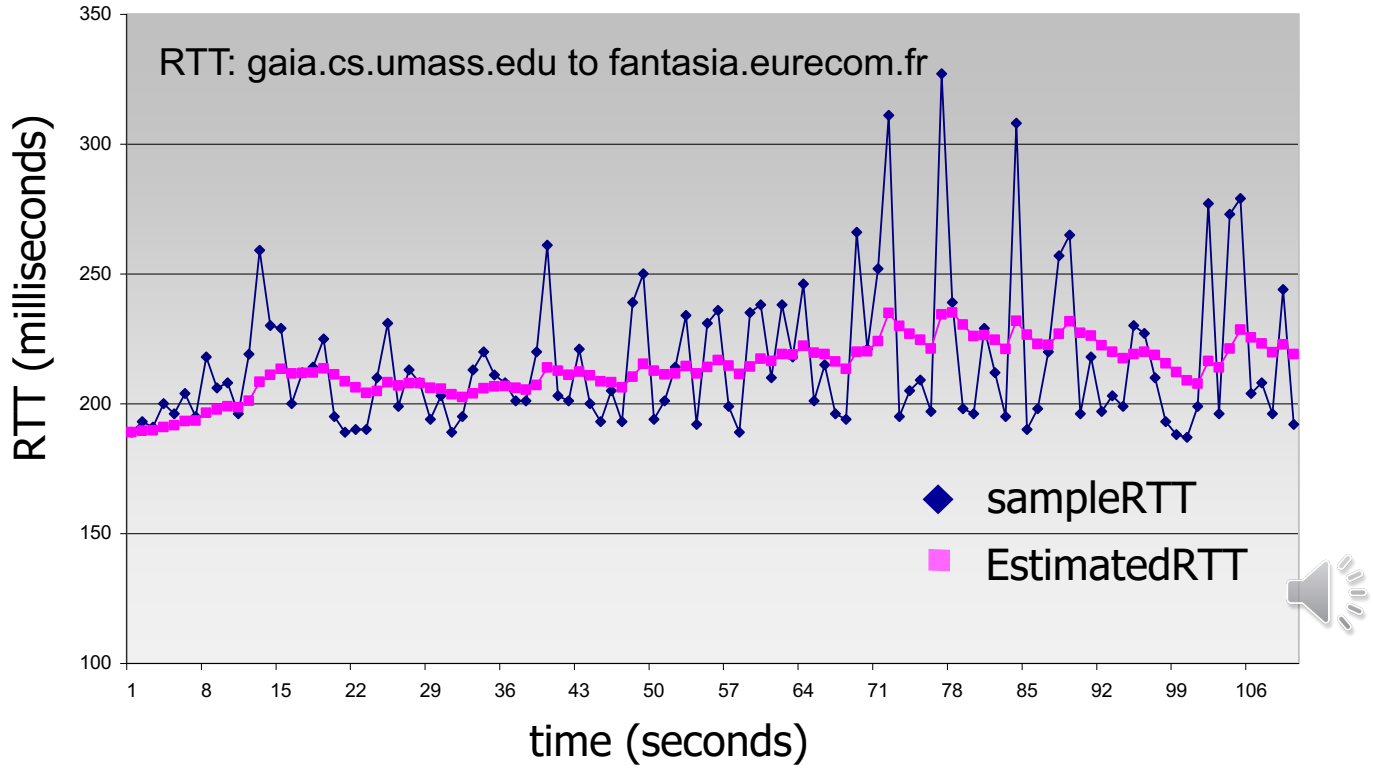
- `SampleRTT`: đo thời gian truyền segment cho đến khi nhận được ACK
  - Bỏ qua việc truyền lại
- `SampleRTT` sẽ khác nhau với mỗi lần đo
  - Tính trung bình nhiều lần đo chứ không chỉ dựa vào lần đo hiện tại



TCP round trip time, timeout

$$\texttt{EstimatedRTT} = (1-\alpha)*\texttt{EstimatedRTT} + \alpha*\texttt{SampleRTT}$$

- Cơ chế EWMA: exponential weighted moving average
- Sự ảnh hưởng của các lần đo trước sẽ giảm theo α
- Giá trị tiêu biểu: $\alpha = 0.125$



TCP round trip time, timeout

- Khoảng timeout: **EstimatedRTT** cộng với "safety margin" – biên độ an toàn"

```
TimeoutInterval = EstimatedRTT + 4*DevRTT
```



estimated RTT

"safety margin"

- **DevRTT**: EWMA of **SampleRTT** deviation from **EstimatedRTT**:

```
DevRTT = (1-β)*DevRTT + β*|SampleRTT-EstimatedRTT|
```

(typically, β = 0.25)

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

TCP Sender (tinh giản)

Sự kiện: Nhận dữ liệu từ tầng ứng dụng

- Tạo segment với STT seq #
- seq #: STT của byte đầu tiên trong segment
- Bắt đầu tính thời gian với segment cũ nhất

Sự kiện: nhận được ACK

- Nếu ACK xác nhận cho các segment chưa được ACK
  - Cập nhật trạng thái
  - Tính lại thời gian với các segment chưa được ACK

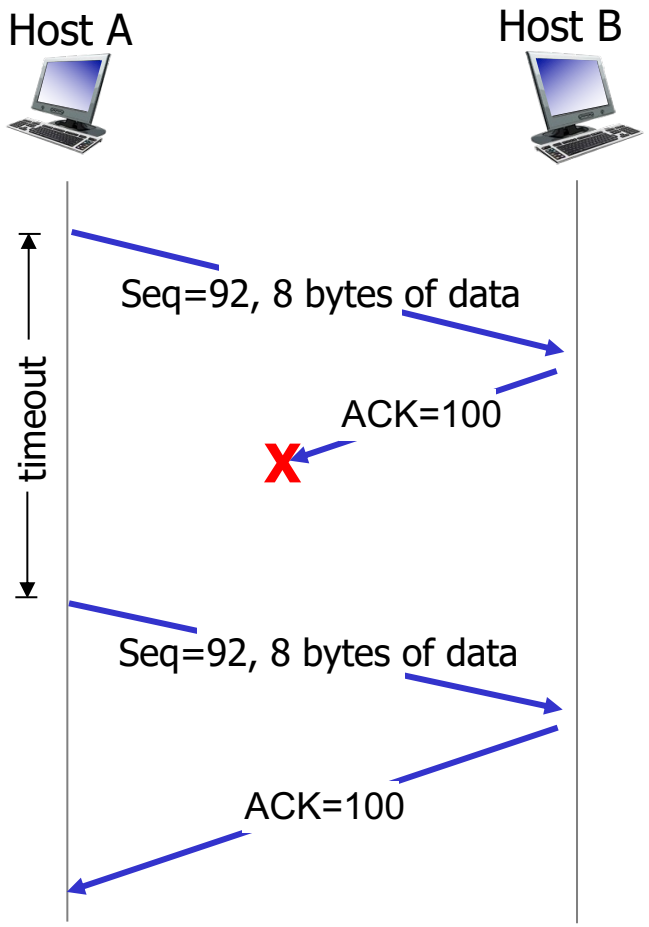
Sự kiện: timeout

- Gửi lại segment bị timeout
- Bắt đầu tính lại thời gian

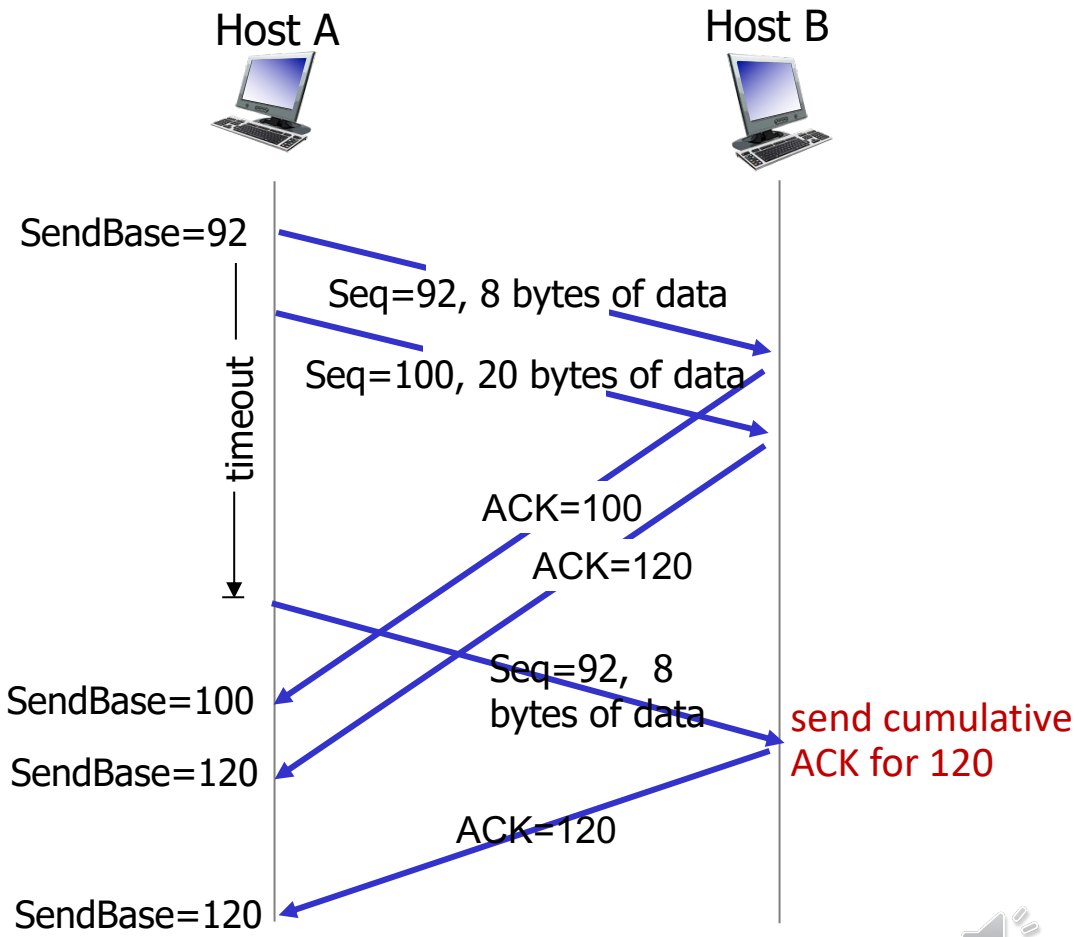
TCP Receiver: ACK generation [RFC 5681]

Sự kiện tại bên nhận	*Hành động*

TCP: các trường hợp truyền lại

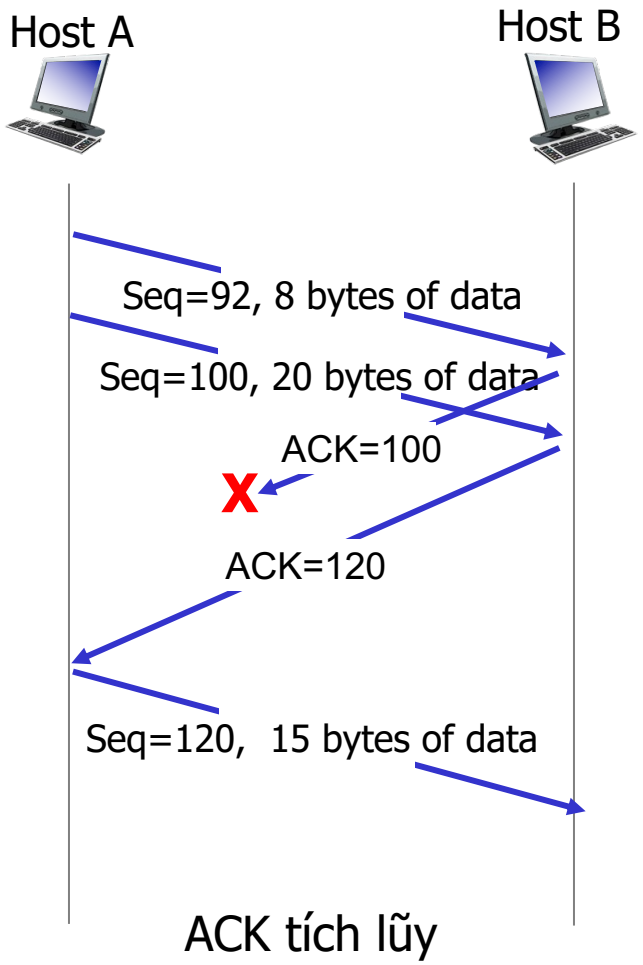


Mất ACK



timeout

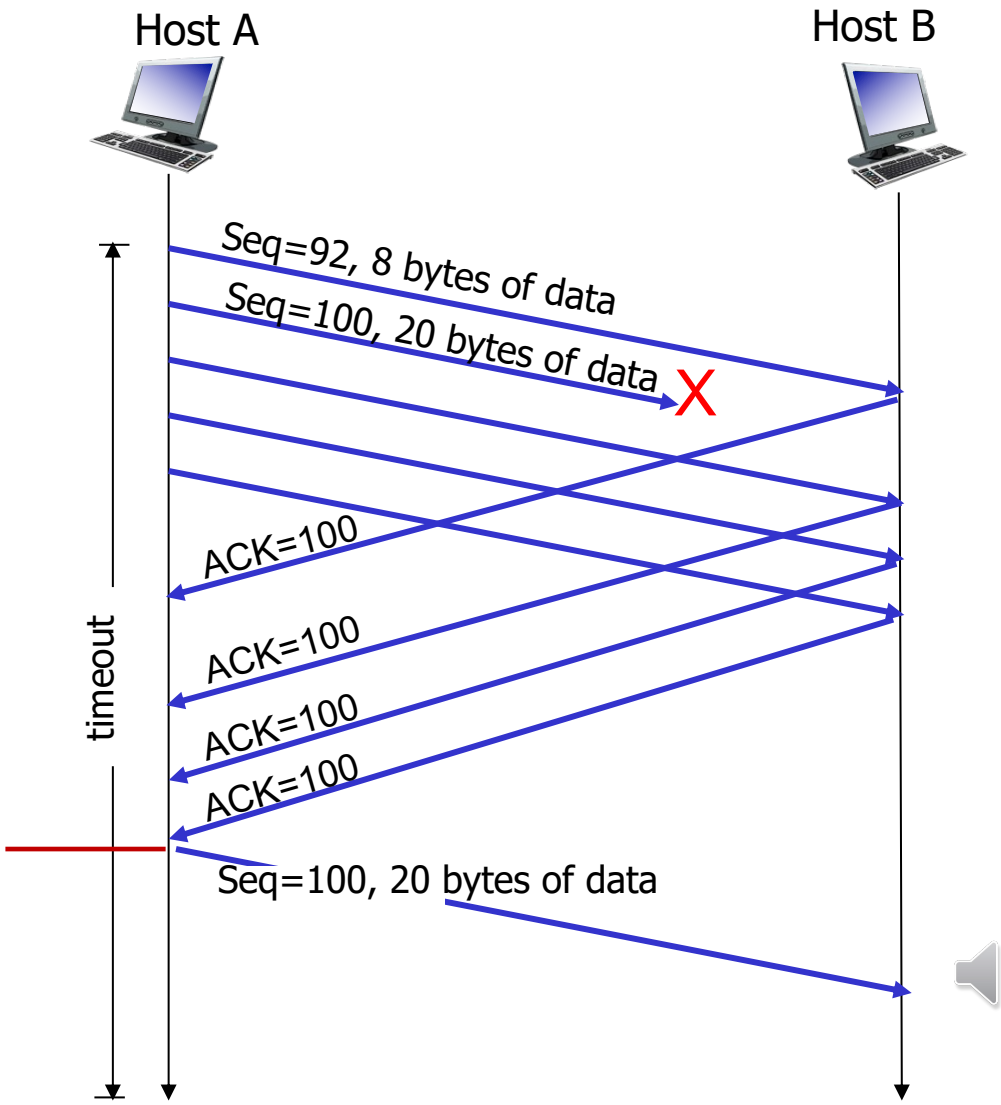
TCP: các trường hợp truyền lại



TCP: truyền lại nhanh

TCP: truyền lại nhanh

Khi bên gửi nhận được 3 ACK yêu cầu cùng segment, bên gửi gửi lại segment được yêu cầu mà không cần chờ timeout.



Việc nhận 3 ACK trùng lặp cho biết đã nhận được 3 segment sau một segment bị thiếu – có khả năng bị mất phân đoạn.
=> Truyền lại!